김지민 iOS Developer

010-3336-1243 jie88min@naver.com

iOS Developer · 6년

운영 중인 앱을 멈추지 않고,

코드는 98% Swift가 됐습니다.

Qoo10 통합 앱 Swift 전환과 신규 쇼핑 앱 3종 출시를 맡았습니다.

사용자가 매일 쓰는 앱을 고치면서도 깨뜨리지 않는 일 — 그게 제가 가장 잘하는 일입니다.

큐텐테크놀로지 2019.05 – 2025.04 **신한대학교** 컴퓨터공학과 학사 jie88min@naver.com

지금까지

한 회사에서 6년, 여러 서비스를 출시부터 운영까지 책임졌습니다

여러 서비스를 맡으며 제가 작성한 코드가 2년 뒤, 4년 뒤 어떤 부담이 되는지까지 직접 겪었습니다. 짧게 거쳐 가서는 알기 어려운 레거시의 맥락과 운영의 무게를 알고, 그래서 **오래 유지될 구조를 먼저 고민**합니다.

앞으로

이제는 더 넓은 문제를 풀고 싶습니다

커머스 앱이 겪는 거의 모든 단계를 지나온 경험을 바탕으로, **더 다양한 도메인**에서 새로운 사용자와 문제를 만나고 싶습니다. "운영 중인 앱을 안전하게 바꾸는 경험"을 새로운 곳에서 다시 증명하겠습니다.

SKILLS

기술

언어 · UI	Swift 주력 · Objective-C 실무 6년 · UIKit · SwiftUI · Auto Layout · SnapKit
비동기 · 구조	async/await · Combine · MVVM · Clean Architecture · Tuist 모듈화
보안 · 인증	LocalAuthentication (Face ID / Touch ID) · Keychain Access Group · AES256
앱 기능	APNs 푸시 · 딥링크 / Universal Link · WKWebView ↔ 네이티브 브리지
빌드 · 배포	SPM · CocoaPods · App Store Connect · Firebase Crashlytics
크로스플랫폼	Flutter 운영·릴리즈 경험 · Dart · Google Play Console

WHAT I BRING

이런 문제가 있는 팀에 바로 도움이 됩니다

ObjC → Swift

레거시가 무거운 앱

Objective-C·XIB가 남은 코드베이스를 기능 단위로 쪼개 Swift·코드 UI로 옮겼습니다. 기존 호출 관계를 유지하고 단계마다 회귀 QA를 거쳐, 로그인·검색 같은 핵심 화면부터 공통 모듈까지 운영 중단 없이 전환을 마쳤습니다.

Tuist 모듈화

여러 앱, 한 코드베이스

Tuist 모듈화와 Clean Architecture로 3개 서비스가 공통 기능을 나눠 쓰는 구조를 만들었습니다. 런처·WebView를 분리 전환해 호환성을 지켰습니다.

Face ID + Keychain

인증과 앱 간 연동

Face ID·Touch ID 로그인과 Keychain Access Group 기반 자사 앱 간 토큰 공유를 설계·구현했습니다. 실패·취소·미지원 흐름까지 검증해 자사 앱 2개에 안정적으로 적용했습니다.

EXPERIENCE

경력

큐텐테크놀로지 · iOS Developer

2019.05 – 2025.04 · 6년

Qoo10 통합 앱을 비롯한 자사 커머스 iOS 앱의 기능 개발·운영·App Store 배포를 재직 기간 내내 담당했습니다. 아래는 그중 주도적으로 맡은 작업입니다.

대표 Qoo10 통합 앱 Objective-C → Swift 전환

2021.03 – 2023.08

Swift 비중 5% → 98%+ · 운영 중단 없이 로그인·통합검색·이미지검색 등 핵심 화면과 공통 모듈을 전환

문제	수년간 쌓인 Objective-C 코드와 공통 로직, Swift 비중 5%. 서비스는 계속 돌아가야 하고 서버 API 변경도 동시에 들어오는 상황.
내가 한 일	• 로그인, 통합검색, 이미지검색, 화면 이동, 이미지 로딩을 **기능 단위로 나눠** 순차 전환

- 이미지검색 화면은 ViewController → SubView → Cell 단위로 쪼개 영향 범위를 줄이며 이관
- XIB 화면을 Auto Layout·SnapKit 코드 UI로 교체
- 앱 전역에서 쓰이는 `MobileAPICommon`, `WebViewOptionManager`, `PriceUtil`, `BarManager`를 정리해 Swift로 이전
- ObjC↔Swift 혼용 구간의 기존 호출 관계를 유지하고, 전환마다 회귀 QA로 기존 기능 동작을 확인

성과 운영 중인 서비스의 기능을 유지하면서 코드 구조와 유지보수성을 개선했습니다. 화면 → 공통 모듈 순으로 단계적으로 전환해, 각 단계를 독립적으로 배포할 수 있었습니다.

Swift · Objective-C · UIKit · SnapKit · Auto Layout · REST API

신규 쇼핑 앱 3종 출시 · Tuist 모듈화

2024.03 – 2024.10

위메프 플러스 · 인팍쇼핑 · 위시플러스 출시 · 6인 팀에서 앱 런처와 WebView 담당

문제 신규 커머스 서비스 3개를 빠르게 출시하면서, 기존 앱의 동작은 유지하고 공통 기능은 재사용해야 했음.

내가 한 일
- Objective-C 앱 런처를 **SwiftUI로 재구현**하고 초기 화면 구성 개선
- 기존 WebView의 생명주기·Delegate 동작을 분석해 신규 서비스용 WebView 설계 및 프로토타입 구현
- Tuist 모듈화와 Clean Architecture 적용, async/await·Combine 기반 비동기·상태 관리 구조 구성
- 기존 서비스 핵심 기능이 신규 앱에서 동일하게 동작하는지 전환 QA 진행

성과 신규 쇼핑 앱 3종을 출시했습니다. 런처와 WebView를 분리해 전환하고 전환 QA를 맡아, 기존 서비스 기능과의 호환성을 확보했습니다.

SwiftUI · Tuist · Combine · async/await · Clean Architecture · WKWebView

---

생체인증 로그인 · 자사 앱 간 로그인 공유

2023.06 – 2024.02

자사 앱 2개 적용 · 한 앱에서 로그인하면 다른 앱에서 토큰 재사용

문제 반복 로그인으로 인한 이탈을 줄이면서, 자사 앱 사이의 인증 상태를 안전하게 연계해야 했음.

내가 한 일
- LocalAuthentication 기반 Face ID·Touch ID 인터페이스 구현 및 기존 로그인 시스템 연동
- **Keychain Access Group**으로 앱 간 저장소를 공유하고, 로그인 토큰은 AES256으로 한 번 더 암호화해 저장
- 인증 실패·사용자 취소·생체인증 미등록·기기 미지원 등 예외 흐름을 시나리오별로 검증

성과 자사 앱 2개에 생체인증 로그인을 적용하고, 한 번 로그인하면 다른 자사 앱에서도 토큰을 재사용하는 구조를 마련해 반복 로그인을 줄였습니다.

LocalAuthentication · Keychain · AES256 · Swift · Objective-C

부가 티몬파트너센터 Flutter 앱 운영 · 릴리즈

2024.03 – 2024.10

내가 한 일
- 8개월간 Flutter 기반 판매자 앱의 iOS·Android 유지보수와 양대 스토어 릴리즈 전담
- QA 후 릴리즈 브랜치 생성 → 빌드 검증 → App Store Connect·Google Play Console 배포
- Xcode 업데이트로 깨진 iOS 빌드와 Flutter·Firebase SDK 호환성 문제 해결

Flutter · Dart · Firebase · App Store Connect · Google Play Console

경력 휴식 · 복귀 준비

2025.05 – 현재

육아로 경력을 쉬는 동안 개인 프로젝트(Maplay)로 설계·개발 감각을 이어가고 있습니다.

HOW I RECHARGE

퇴근 후의 취미가, 45만 명이 쓰는 서비스가 됐습니다

개발을 오래 잘하려면, 일을 잘하는 것만큼 **스트레스를 어떻게 푸는지**도 중요하다고 생각합니다.

저는 게임을 좋아합니다. 게임을 하다가 "이 정보를 한눈에 볼 수 있으면 좋겠다"는 생각이 들어 직접 만들었습니다. 그렇게 시작한 **던파OFF**는 4년 반 동안 운영하며 **MAU 45만+** 서비스가 됐고, 경력을 잠시 쉬는 지금도 개발을 놓지 않기 위해 **Maplay**를 만들며, 실무에서 쌓은 설계·안정성 경험을 이어가고 있습니다.

제게 게임은 머리를 식히는 방법이면서, 회사에서는 겪기 어려운 문제를 먼저 겪어 보는 실험실이었습니다. 좋아하는 일로 쉬기 때문에 지치지 않고, 그 안에서 배운 것을 다시 업무로 가져올 수 있었습니다.



홈 · 검색



캐릭터 상세



채팅

SIDE PROJECTS

사이드 프로젝트

MAU 45만+ **던파OFF**

2019.02 – 2023.09 · 서비스 종료

게임 '던전앤파이터' 캐릭터 정보·순위 조회 서비스 (Web · iOS · Android). **3인 팀 리드**로 모바일 앱 개발·스토어 배포와 팀 일정·우선순위 조율을 맡았습니다.

네트워크가 흔들려도 앱은 흔들리지 않게 — 화면 상태 설계

이벤트 시점 트래픽 급증, 순위 조회 실패, 외부 API 호출 한도 초과를 겪으며, 요청 결과를 사용자가 이해할 수 있는 화면 상태로 연결했습니다.

상황	앱에서 한 처리	서버 쪽 대응
요청 폭주로 응답 지연	캐시된 데이터를 먼저 보여주고 로딩 상태를 별도 표시	부하 구간 분석, 처리 용량 확장
순위 조회 실패	실패를 빈 화면이 아닌 오류 상태로 노출, 재시도 제공	조회 병목 분석, DB 쿼리 튜닝
외부 API 한도 초과	"최신 조회 실패"와 "기존 데이터 표시"를 분리, 안내 + 재시도	API 호출·데이터 처리 구조 조정

iOS · Android · Cache · Error / Retry UX · Spring · MariaDB

Maplay

2026.04 – 진행 중 · 1인

넥슨 Open API로 메이플스토리 캐릭터의 장비·능력치를 수집해 전투 성능을 계산하고, 장비를 바꿨을 때의 변화를 미리 비교해 주는 웹 서비스. 경력 휴식 중 혼자 설계·개발하고 있습니다.

구조 — 20여 종 외부 API → **13개 독립 수집 모듈** → 계산 파이프라인. 50개+ 응답을 DTO로 타입화하고, 일부 API가 실패해도 나머지 결과는 정상 표시.

안정성
- 요청당 10초 타임아웃 · 일시 오류 1회 재시도 + 지터 · 실패 시 마지막 성공 데이터를 기준 시점과 함께 표시
- 동일 캐릭터 동시 요청은 키별 Promise 공유로 계산 1회만 수행, 부분 실패 결과는 캐시하지 않음
- CPU 계산을 Worker Thread 풀로 격리, 30초 초과 작업은 워커 교체 후 재배정

검증 — 비공개 계산 공식을 실제 게임 화면 수치와 대조해 역추정하고 오차를 보정.

iOS로 옮기면: 타임아웃·재시도·stale 데이터 표시는 `URLSession` + `async/await`로, 동일 요청 공유는 `actor` 기반 in-flight Task 캐시로 그대로 대응됩니다.

TypeScript · NestJS · Next.js · Monorepo